

DIA 7 — Resoluções dos Desafios SQL de 1 a 8

Introdução

No capítulo anterior, apresentamos uma série de desafios criados para praticar os principais fundamentos da linguagem SQL.

Agora chegou o momento de analisar as resoluções dos oito primeiros exercícios.

Mais importante do que simplesmente copiar os comandos é compreender o raciocínio utilizado em cada consulta. Durante as resoluções, trabalharemos com:

- Seleção de colunas;
- Filtros com `WHERE` ;
- Ordenação com `ORDER BY` ;
- Limitação de resultados com `LIMIT` ;
- Relacionamentos com `JOIN` ;
- Subconsultas;
- Cálculos matemáticos;
- Funções de agregação;
- Agrupamentos com `GROUP BY` .

Exercício 1 — Listar todos os clientes cadastrados

O primeiro desafio solicita a listagem de todos os clientes cadastrados, mostrando apenas o nome e o sobrenome.

```
SELECT
    nome_cliente,
    sobrenome_cliente
FROM clientes;
```

Entendendo a consulta

A cláusula `SELECT` informa quais colunas devem ser apresentadas no resultado:

```
nome_cliente,
sobrenome_cliente
```

Já a cláusula `FROM` indica de qual tabela essas informações serão recuperadas:

```
FROM clientes
```

Embora fosse possível utilizar `SELECT *`, não precisamos retornar todas as colunas da tabela. Como o exercício solicita apenas o nome e o sobrenome, selecionamos somente essas informações.

Essa prática torna a consulta mais clara e evita o retorno de dados desnecessários.

Exercício 2 — Listar produtos com preço maior que R\$ 20,00

O segundo exercício solicita todos os produtos cujo preço seja superior a R\$ 20,00.

```
SELECT
    nome_produto,
    preco
FROM produtos
WHERE preco > 20.00;
```

Entendendo a consulta

A cláusula `WHERE` é responsável por aplicar um filtro aos registros.

```
WHERE preco > 20.00
```

O operador `>` significa **maior que**.

Portanto, produtos com preço exatamente igual a R\$ 20,00 não serão retornados. A consulta considera apenas valores superiores a esse limite.

Caso o exercício solicitasse produtos com preço maior ou igual a R\$ 20,00, utilizaríamos:

```
WHERE preco >= 20.00
```

Exercício 3 — Ordenar os produtos do mais caro para o mais barato

Neste exercício, precisamos listar os produtos em ordem decrescente de preço.

```
SELECT
    nome_produto,
    preco
FROM produtos
ORDER BY preco DESC;
```

Entendendo a consulta

A cláusula `ORDER BY` é utilizada para ordenar os registros retornados.

```
ORDER BY preco DESC
```

A palavra `DESC` indica uma ordenação decrescente. Dessa forma, os maiores preços aparecem primeiro.

A sequência do resultado será semelhante a:

```
50.00
45.00
30.00
18.00
11.00
...
```

Caso utilizássemos `ASC`, a ordenação seria crescente, começando pelo produto mais barato.

```
ORDER BY preco ASC
```

Como `ASC` é o comportamento padrão, também poderíamos escrever apenas:

```
ORDER BY preco
```

Exercício 4 — Mostrar os três produtos mais baratos

Agora precisamos combinar ordenação e limitação de resultados.

```
SELECT
    nome_produto,
    preco
```

```
FROM produtos
ORDER BY preco ASC
LIMIT 3;
```

Entendendo a consulta

Primeiro, os produtos são ordenados do menor preço para o maior:

```
ORDER BY preco ASC
```

Depois, o banco limita o resultado aos três primeiros registros:

```
LIMIT 3
```

A ordem dessas operações é importante.

O banco primeiro organiza os resultados e, em seguida, seleciona apenas os três primeiros. Como a ordenação é crescente, esses registros representam os três produtos mais baratos.

Sem o `ORDER BY`, o banco poderia retornar quaisquer três registros, sem garantir que fossem realmente os mais baratos.

Exercício 5 — Listar os pedidos com cliente, produto e quantidade

As informações solicitadas estão distribuídas em três tabelas diferentes:

- O nome do cliente está na tabela `clientes`;
- O nome do produto está na tabela `produtos`;
- A quantidade comprada está na tabela `pedidos`.

Para reunir essas informações, utilizamos `JOIN`.

```
SELECT
    c.nome_cliente,
    c.sobrenome_cliente,
    p.nome_produto,
    pe.qtde
FROM pedidos pe
JOIN clientes c
    ON pe.cod_cliente = c.cod_cliente
JOIN produtos p
    ON pe.cod_produto = p.cod_produto;
```

Entendendo os aliases

Para deixar a consulta mais curta e legível, criamos apelidos para as tabelas:

```
pedidos pe  
clientes c  
produtos p
```

Assim, em vez de escrever:

```
clientes.nome_cliente
```

podemos utilizar:

```
c.nome_cliente
```

Os aliases não modificam o nome verdadeiro das tabelas. Eles existem apenas durante a execução da consulta.

Relacionando pedidos e clientes

O primeiro `JOIN` conecta a tabela `pedidos` à tabela `clientes`.

```
JOIN clientes c  
ON pe.cod_cliente = c.cod_cliente
```

A condição determina que o código do cliente armazenado no pedido deve ser igual ao código existente na tabela de clientes.

Por exemplo:

```
pedidos.cod_cliente = 1  
clientes.cod_cliente = 1
```

Quando os valores correspondem, o banco compreende que aquele pedido pertence àquele cliente.

Relacionando pedidos e produtos

O segundo `JOIN` conecta os pedidos aos produtos.

```
JOIN produtos p
ON pe.cod_produto = p.cod_produto
```

A coluna `cod_produto` da tabela `pedidos` é comparada com a chave primária da tabela `produtos`.

Com isso, conseguimos apresentar em um único resultado informações que originalmente estavam separadas.

Exercício 6 — Listar os pedidos utilizando subconsulta

O exercício 6 solicita as mesmas informações do exercício anterior, mas exige que o nome do cliente seja recuperado por meio de uma subconsulta.

```
SELECT
(
    SELECT CONCAT(c.nome_cliente, ' ', c.sobrenome_cliente)
    FROM clientes c
    WHERE c.cod_cliente = pe.cod_cliente
) AS cliente,
p.nome_produto,
pe.qtde
FROM pedidos pe
JOIN produtos p
ON p.cod_produto = pe.cod_produto;
```

Entendendo a subconsulta

Uma subconsulta é uma consulta executada dentro de outra consulta.

Neste caso, a consulta interna é:

```
SELECT CONCAT(c.nome_cliente, ' ', c.sobrenome_cliente)
FROM clientes c
WHERE c.cod_cliente = pe.cod_cliente
```

Ela procura o cliente cujo código corresponde ao código armazenado no pedido.

A função `CONCAT` junta o nome e o sobrenome em uma única informação:

```
CONCAT(c.nome_cliente, ' ', c.sobrenome_cliente)
```

O resultado será semelhante a:

```
João Silva  
Maria Oliveira  
Pedro Santos
```

A coluna calculada recebe o alias `cliente`:

```
AS cliente
```

Embora essa solução seja válida para fins de aprendizado, normalmente o `JOIN` é mais apropriado para conectar tabelas relacionadas.

O objetivo deste exercício é demonstrar que uma consulta pode utilizar o resultado produzido por outra consulta.

Exercício 7 — Listar produtos com menos de 20 unidades em estoque

Neste exercício, precisamos identificar produtos com estoque baixo.

```
SELECT  
    nome_produto,  
    qtde_estoque  
FROM produtos  
WHERE qtde_estoque < 20;
```

Entendendo a consulta

O filtro considera somente os produtos cuja quantidade disponível seja inferior a 20:

```
WHERE qtde_estoque < 20
```

Produtos com exatamente 20 unidades não serão exibidos.

Caso a regra fosse “estoque menor ou igual a 20”, utilizaríamos:

```
WHERE qtde_estoque <= 20
```

Esse tipo de consulta pode ser utilizado em sistemas comerciais para gerar alertas de reposição, relatórios de estoque crítico ou listas automáticas de compra.

Exercício 8 — Mostrar o valor total gasto por João Silva

Este exercício exige a combinação de relacionamentos, filtros, cálculos, agregações e agrupamentos.

```
SELECT
    c.nome_cliente,
    c.sobrenome_cliente,
    SUM(pr.preco * pe.qtde) AS valor_total
FROM pedidos pe
JOIN clientes c
    ON pe.cod_cliente = c.cod_cliente
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
WHERE c.nome_cliente = 'João'
    AND c.sobrenome_cliente = 'Silva'
GROUP BY
    c.nome_cliente,
    c.sobrenome_cliente;
```

Relacionando as tabelas

Primeiro, conectamos os pedidos aos clientes:

```
JOIN clientes c
    ON pe.cod_cliente = c.cod_cliente
```

Depois, conectamos os pedidos aos produtos:

```
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
```

Essas relações são necessárias porque:

- A identificação do cliente está na tabela `clientes`;
- A quantidade comprada está na tabela `pedidos`;
- O preço está na tabela `produtos`.

Filtrando o cliente

A cláusula `WHERE` restringe os resultados ao cliente João Silva.


```
WHERE c.nome_cliente = 'João'  
AND c.sobrenome_cliente = 'Silva'
```

O operador `AND` determina que as duas condições precisam ser verdadeiras simultaneamente.

Essa separação entre nome e sobrenome é necessária porque essas informações estão armazenadas em colunas diferentes.

Calculando o valor de cada pedido

Para descobrir o valor de um item comprado, multiplicamos seu preço pela quantidade:

```
pr.preco * pe.qtde
```

Por exemplo, se um produto custa R\$ 45,00 e foram compradas duas unidades:

```
45,00 × 2 = 90,00
```

Somando todos os valores

Como João pode possuir vários pedidos, utilizamos a função `SUM`:

```
SUM(pr.preco * pe.qtde)
```

Ela soma o valor de todos os produtos comprados por ele.

O resultado recebe o alias:

```
AS valor_total
```

Agrupando os resultados

Como estamos utilizando uma função de agregação junto com colunas comuns, precisamos adicionar essas colunas ao `GROUP BY`.

```
GROUP BY
  c.nome_cliente,
  c.sobrenome_cliente
```

Isso informa ao banco que todos os pedidos do mesmo cliente devem fazer parte de um único grupo.

Ao final, João Silva aparece apenas uma vez, acompanhado pelo valor total gasto.

Exercício adicional — Mostrar a quantidade total de clientes cadastrados

O último comando enviado calcula quantos clientes estão cadastrados no banco de dados.

```
SELECT
  COUNT(*) AS total_clientes
FROM clientes;
```

Entendendo a função COUNT

A função `COUNT` é utilizada para contar registros.

```
COUNT(*)
```

O asterisco indica que todas as linhas da tabela devem ser consideradas.

Se a tabela possuir cinco clientes, o resultado será:

```
total_clientes
-----
5
```

O alias:

```
AS total_clientes
```

torna o nome da coluna mais claro e compreensível.

Sem o alias, o banco normalmente apresentaria apenas `count` como nome da coluna.

Na lista original de desafios, a contagem total de clientes corresponde ao exercício 9. Por isso, ela foi apresentada aqui como exercício adicional, depois das resoluções de 1 a 8.

Resumo das Resoluções

Os primeiros desafios trabalharam consultas simples:

```
SELECT  
WHERE  
ORDER BY  
LIMIT
```

Em seguida, avançamos para relacionamentos entre tabelas:

```
JOIN  
ON
```

Também utilizamos uma subconsulta para recuperar informações relacionadas:

```
SELECT (  
    SELECT ...  
)
```

Por fim, realizamos cálculos e agregações com:

```
SUM  
COUNT  
GROUP BY
```

Conclusão

As primeiras resoluções demonstram como diferentes recursos da linguagem SQL podem ser combinados para responder perguntas reais.

Começamos com consultas simples, selecionando e filtrando registros. Depois, aprendemos a ordenar e limitar resultados. Em seguida, conectamos tabelas com `JOIN`, recuperamos informações com subconsultas e calculamos valores utilizando funções de agregação.

O ponto mais importante é entender que consultas complexas podem ser construídas gradualmente.

Primeiro identificamos as tabelas necessárias. Depois criamos os relacionamentos, aplicamos os filtros, realizamos os cálculos e, por fim, agrupamos ou ordenamos os resultados.

Essa forma de raciocínio será fundamental para resolver desafios SQL cada vez mais avançados.